```
def generateTableFromClf(self):
    featureNames = [self.dfFeature[i] for i in self.clf.tree_.feature]
    leafIds = self.clf.apply(self.dfData)
    leftChildren = self.clf.tree_.children_left
    rightChildren = self.clf.tree_.children_right
    decPath = self.clf.decision_path(self.dfData)
    threshold = self.clf.tree_.threshold

    featuresForDiagram = list(self.dfFeature)
    featuresForDiagram.append(self.dfClassColumn)

    self.clearDecisionTable()
    self.generateTableColumns(featuresForDiagram)

    bar = IncrementalBar("Processing",suffix='%(percent)d%%', max = len(set(leafIds)))
    for i in set(leafIds):
        samplesInNode = decPath.getcol(i).copy()
        rows,cols = samplesInNode.nonzero()[0]
        sampleId = rows[0]
        nodeIndex = decPath.indices[decPath.indptr[sampleId]:decPath.indptr[sampleId+1]]
        className = self.clf.classes_[np.argmax(self.clf.tree_.value[i])]
        inputOutput = defaultdict(dict)
        for value in featuresForDiagram[:-1]:
                inputOutput[className][value] = {}
        for index, nodeId in enumerate(nodeIndex):
            nodeFeature = featureNames[nodeIndex[index-1]]
            nodeThreshold = threshold[nodeIndex[index-1]]
            if nodeId in set(leftChildren):
                inputOutput[className][nodeFeature]["<="] = nodeThreshold
            if nodeId in set(rightChildren):
                inputOutput[className][nodeFeature][">"] = nodeThreshold
        self.generateTableRows(inputOutput.items())
        bar.next()
    bar.finish()
    self.writeTree()
```